NDYRA — Soup-to-Nuts Blueprint

Aelric Agent Edition • Infra + App Build (Consolidated)

Version: v7.4 (Consolidated)

Build State Reference: CP75 (2026-03-02_75)

Published: 2026-03-03

Owner: William Davis Moore

Audience: Aelric Architect + Aelric Agent (Execution)

Status: Approved blueprint (treat as engineering law)

Table of Contents

1. 0. Governance: Anti-Drift + IP Guardrails
2. 1. System Overview: What NDYRA is (and is not)
3. 2. Product Surfaces + Route Map (Public / App / Biz / Admin)
4. 3. UI System: Emulation rules, layout grammar, icon language, gesture law
5. 4. Social Layer: Posts, Feeds, Signals, Aftermath, Sharing
6. 5. Gym Layer: Connected Gym, Constellation Rating, Members, Events, Announcements
7. 6. Competitive Layer: Seasons, Challenges, Streaks, Shields, Trophies
8. 7. Messaging Layer: Inbox, Announcements, DMs, Privacy, Safety
9. 8. Ops Layer: Waivers, Tokens, Booking, Check-in, Migration
10. 9. Data Model + Security: Tables, enums, RPCs, RLS contracts
11. 10. Infra Wiring: Git, Supabase, Netlify, Stripe (step-by-step)
12. 11. QA Gates + Release Discipline
13. Appendix A: Environment Variables + Secrets Map
14. Appendix B: Netlify redirects/headers requirements
15. Appendix C: Known drifts found in CP75 + required fixes

0. Governance: Anti-Drift + IP Guardrails

NDYRA is being built under an anti-drift regime. The goal: every new surface must have one authoritative source of truth (schema + routes + security), and we do not allow silent divergence between UI and backend rules.

0.1 Canonical sources of truth (must stay aligned)

- Repo schema is law: /supabase/migrations/*.sql (ordered by filename).
- RLS is law: no permissive policies; post-adjacent reads must gate through can_view_post(post_id).
- Locked brand tokens are law: black canvas + white type + NDYRA red accent (#E10600).
- Routes are law: /site/* plus /site/_redirects must cover dynamic routes exactly.
- QA gates are law: qa_smoke, qa_super, IP gate, Supabase anti-drift audit, Supabase RLS regression tests.

0.2 No parallel stacks

- No separate “demo social system” and “real social system.” Demo mode is allowed only as a UI seed; it must reuse the same components/routes/contracts as real mode.
- No separate media pipeline for Signals/Aftermath. Reuse post_media + Storage strategy.
- No new UI frameworks inside this repo. Keep the build as static HTML + ES modules + CSS tokens.

0.3 IP Guardrails (law before merge)

NDYRA must stay distinctive. Emulation is allowed at the level of structure (layout grammar, interaction patterns) but never pixel-copying, asset copying, or copywriting mimicry. High-risk areas include Signals, music, GIFs/stickers, beauty filters, biometric reveals, and booking UX.

- Never ship unlicensed music catalogs, third-party sticker packs, fonts, or copied icons.
- Signals must avoid “timed auto-advance gallery as the defining behavior.” Prefer manual swipe navigation and NDYRA-native UI grammar.
- Aftermath must stay single-user, single-session, privacy-first; no multi-user biometric comparisons.

0.4 Definition of done for any checkpoint

- Schema + RLS updated (if needed) before UI relies on it.
- Netlify functions updated (if needed) with least-privilege secrets and correct routing.
- Local QA runs clean (start:qa) and passes qa:all (or documented exceptions).
- Artifacts packaged cleanly (no loose patch zips; full repo replace + master parts).

1. System Overview: What NDYRA is (and is not)

1.1 Product thesis

NDYRA is a social-first fitness platform that stitches together three realities:

- Member reality: train, recover, share, compete, connect.
- Gym reality: run sessions, manage members, publish events/announcements, collect feedback, retain.
- Platform reality: enforce privacy + safety by default, and make progress measurable without turning the app into negativity.

Vibe target: Hardcore competitor energy (rank, rare badges, season trophies) fused with Zen habit energy (streaks, consistency rewards, gentle nudges, clean shareables).

1.2 Architecture snapshot (build to date)

- Frontend: static site in /site (HTML pages + ES modules + CSS). Runs as a PWA with service worker + offline shell.
- Backend: Supabase Postgres (RLS + RPCs) + Storage + Realtime presence.
- Serverless: Netlify Functions for Stripe + migration utilities + telemetry + public config.
- Deployment: Netlify hosts /site and serves functions under /.netlify/functions/* plus /api/* rewrite.

1.3 Non-goals (explicit)

- No competitor “clone” UX. Emulate only what is structurally useful; keep NDYRA visuals and grammar original.
- No hardcoded master credentials in the repo.
- No client-side security assumptions. Anything sensitive must be enforced in Supabase (RLS/RPC).

2. Product Surfaces + Route Map (Public / App / Biz / Admin)

2.1 Public marketing + entry

Public pages are accessible without auth and should never block QA navigation. They must remain lightweight and brand-correct.

- / (routes to Social Shell by default; marketing preview lives at /preview/)
- /pricing.html, /for-gyms/* (gym sales surface)
- /join.html, /login.html, /auth/login.html (auth entry points)
- /gym/:slug (public gym profile)
- /gym/:slug/join (Quick Join flow)
- /workouts/* (public workouts surface / marketing)

2.2 Member app routes (static pages + dynamic rewrites)

Member app routes live under /app/. The AppShell provides the frame (nav + rails) and each route injects its content. Dynamic segments are served through /site/_redirects rewrites.

Route	Surface	Auth	Notes
/app/fyp/	For You feed	Optional	Demo mode allowed; real mode uses Supabase posts + score.
/app/following/	Following feed	Optional	Uses follows_users + follows_tenants in real mode.
/app/search/	Search	Optional	People + gyms + content (progressive enhancement).
/app/create/	Composer	Required for posting	Create post/signal/event; staff tools gated.
/app/post/:id	Post detail	Optional	Dynamic rewrite → /app/post/index.html.
/app/profile/:id	Profile (viewer)	Optional	Dynamic rewrite; respects privacy + blocks.
/app/u/:handle	Profile alias	Optional	Handle resolver → /app/u/index.html.
/app/signals/:id?	Signals viewer	Optional	Muted by default;

			manual navigation.
/app/aftermath/	Aftermath hub	Optional	Viewer + share surfaces (no comparisons).
/app/gyms/	Gyms + Connected Gym	Optional	Public data + member gates for actions.
/app/members/	Members directory	Auth required	Connected Gym scoped; can_message computed server-side.
/app/challenges/	Seasons + challenges	Auth required	Join, log, leaderboard, recap, streak + shields.
/app/trophies/	Trophy cabinet	Auth required	Visibility controls: private/followers/public.
/app/events/	Events	Auth required	List + detail + RSVP + share + .ics export.
/app/inbox/	Inbox	Auth required	Announcements + DMs + requests (tabs).
/app/notifications/	Notifications	Auth required	System + social + messages; badge share links.
/app/check-in/	Check-in	Auth required	Membership/token gate; staff override via Biz.
/app/book/class/:id	Class booking	Auth required	Membership vs tokens fork.
/app/timer/	Timer hub	Optional	Timer is separate pillar; legacy kept at /prototype/.
/app/account/	Account + privacy	Auth required	Toggles: online status, trophies, DM allow, streak nudges.
/app/shop/	Shop	Optional	Placeholder / future commerce.
/app/workouts/	Workouts	Optional	Library + share hooks.

/app/	App router shell	Optional	Fallback rewrite target for unknown /app/* paths.

Dynamic routes are served via Netlify-style rewrites in /site/_redirects. Local QA server must match those rewrites exactly.

2.3 Business portal (gym staff)

Biz routes live under /biz/. These require a Supabase session plus tenant role rows (tenant_users).

Route	Surface	Notes
/biz/onboarding/	Gym onboarding	Creates/claims tenant; sets basics.
/biz/dashboard/	Biz dashboard	Constellation summary + trend + ops visibility.
/biz/settings/	Biz settings	Kill switches + staff tools.
/biz/check-in/	Staff check-in	Override + operational check-in.
/biz/migrate/*	Migration hub	CSV import, verify, commit, cutover.
/biz/gym-timer/*	Gym timer tools	Programming + templates.
/biz/account/	Billing + plan	Stripe portal entry; plan visibility.
/biz/	Biz router shell	Fallback rewrite for unknown /biz/*.

2.4 Admin portal (platform)

Admin routes live under /admin/. These require platform_admins membership.

Route	Surface	Notes
/admin/status/	Status + QA tools	Build label, env sanity, cache clear tools.
/admin/tenants/	Tenants	Platform view of tenants.
/admin/coupons/	Coupons	Stripe coupons mapping + NDYRA coupons.
/admin/comps/	Comps	Admin overrides; no free-for-all.

/admin/	Admin router shell	Fallback rewrite for unknown /admin/*.

3. UI System: Emulation rules, layout grammar, icon language, gesture law

3.1 Emulation rules (structure, not cloning)

- Allowed: structural inspiration (three-column shell, post cards, rails, icon-first nav).
- Banned: pixel-perfect replication, copied icons, copied microcopy, copied motion signatures, copied audio UI.
- NDYRA should feel premium-athletic: black canvas, red energy, clean white type, glass panels, minimal chrome.

3.2 Layout grammar (AppShell)

The AppShell is the frame for all /app routes. It must behave consistently across surfaces.

- Desktop: 3-column grid → Left Nav (fixed) | Center Feed (max 680–760px) | Right Rail (context).
- Mobile: Topbar (optional) + Center | Bottom Nav (Home, Gyms, Create, Timer, Profile).
- Center column owns scroll; rails are sticky and non-janky.
- All pages must render inside the shell without shifting the nav or breaking spacing.

3.3 Nav order (member app)

Nav order is an explicit product decision; do not drift.

- For You
- Following
- Gyms
- Challenges
- Messages (Inbox)
- Signals
- Aftermath
- Members
- Events
- Shop

3.4 Icon language (NDYRA visual grammar)

Icons are not decoration; they are navigational language. Use NDYRA icon packs, never emoji as primary nav metaphors.

- Inactive state: /assets/iconpack/svg_mono/* (monochrome).
- Active state: /assets/iconpack/svg/* (full color).
- Badges/Trophies: /assets/badges/svg/* and /assets/badges/svg_mono/*.
- Icons should always be wrapped in an icon-chip so they remain legible at small sizes.

Nav Item	Icon Key	Notes
For You	spark	

Following	star	
Gyms	gym	
Challenges	trophy	
Messages	message	
Signals	signal	
Aftermath	aftermath	
Members	members	
Events	calendar	
Shop	shop	
Create	+	(special FAB / plus)
Timer	timer	
Profile	profile	

3.5 Gesture law (feed cards)

- Single tap: open post detail.
- Double tap: quick react (default reaction = fire).
- Tap-and-hold: reveal Aftermath overlay while held (no ‘sticky’ hold state unless explicitly designed).
- No accidental scroll hijacking. Interaction must never block scroll.

3.6 Tone system (Hardcore × Zen, in one language)

NDYRA must serve both archetypes without whiplash. We do it by separating visuals (theme variants) from semantics (the same underlying truth).

- Hardcore theme: rank, podium, trophies, ‘earned’ language, rare badges, season banners.
- Zen theme: streaks, days active, gentle nudges, calm gradients, ‘show up’ language.
- Share templates always offer both variants from the same data payload.

4. Social Layer: Posts, Feeds, Signals, Aftermath, Sharing

4.1 Posts: core contract

All social surfaces are built on one primitives set: posts + media + reactions + comments + stats. Do not fork.

- posts.author_user_id OR posts.author_tenant_id (exactly one).
- posts.visibility: public | followers | members | staff | private (enforced via can_view_post).
- posts.kind: post | signal | aftermath (enum; keep controlled).
- post_media: references posts; storage_path points to Supabase Storage.
- post_stats: denormalized counters; refreshed by triggers/RPCs.
- post_reactions + post_comments are RLS-gated and must respect can_comment_now (throttle).

4.2 Feeds

- For You feed is score-driven and time-decayed. It should feel alive but not random.
- Following feed is identity-driven (people + gyms you explicitly follow).
- Seek pagination only (created_at cursor or score cursor). No offset pagination on primary feeds.

For You (FYP)

- Client fetches recent posts and ranks by post_score (materialized/updated by refresh_post_score).
- Demo mode allowed for deterministic QA; demo content must still follow the PostCard contract.

Following

- Canonical source: get_following_feed(limit, cursor) RPC (SECURITY INVOKER).
- Visibility still enforced by can_view_post; RPC must never widen scope.

4.3 Signals (Disciplined stories)

Signals are implemented as posts.kind='signal' with expiry and strict quotas. Signals are muted by default; the user taps to hear.

- Expiry: expires_at must be within 24h of created_at unless pinned_to_profile=true.
- Quotas: 2 active signals per user; 10 active signals per tenant (enforced by enforce_signal_limits trigger).
- Visibility: must reuse can_view_post (signals do not get a special permissive policy).
- Navigation: manual swipe, no signature competitor progress-bar experience as the defining behavior.

4.4 Aftermath (NDYRA-native recap)

Aftermath is a recap card that can be posted as its own post kind (posts.kind='aftermath'). It is a single-user summary for a single session.

- Allowed: heart rate zones, duration, rounds, calories (if available), subjective note.
- Banned: multi-user comparisons, leaderboards embedded in the overlay, default-public biometrics.
- Privacy: must follow post visibility + privacy settings; never auto-publicize biometrics.

4.5 Sharing (retention loop)

Sharing is not a bolt-on; it’s the retention loop. Every major module produces a recap card that is shareable.

- Share templates are 1080×1920 (IG story-ready) with Hardcore + Zen variants.
- Templates must be generated client-side (canvas) from a stable data payload.
- Share never leaks private data; template generator must respect visibility.

5. Gym Layer: Connected Gym, Constellation Rating, Members, Events, Announcements

5.1 Tenants and the 'Connected Gym' concept

- tenants is the gym entity. It owns schedule, memberships, tokens, and public profile content.
- Each user can optionally set one Connected Gym for contextual features (Active Now, Members, Events, Inbox).
- Storage location: privacy_settings.connected_tenant_id with timestamp connected_updated_at.
- Setter RPC: set_connected_tenant(tenant_id) enforces membership/eligibility rules.

5.2 Active Now (presence)

- Presence is scoped to Connected Gym only (no global online list).
- Respect privacy_settings.show_online_status; default should be conservative.
- Realtime uses Supabase presence channels; do not leak user IDs to non-members.

5.3 Constellation Rating (verified-only, aggregated safe)

Constellation is a gym rating system designed for usefulness, not drama.

- Verified-only gate: can_rate_tenant(tenant_id) = true only if user has verified membership within last 60 days.
- gym_ratings holds one current rating per (tenant_id, user_id).
- gym_rating_history is immutable snapshots on insert/update.
- gym_rating_summary is the only table safe for broad reads (aggregates only).
- Optional note field exists but must be treated as 'member feedback' not public review content by default.

Constellation schema contract

Table	Purpose	Read surface
gym_ratings	Canonical per-user rating row (moderated)	Member-only (owner + staff + admin)
gym_rating_history	Immutable audit trail of rating changes	Staff/admin only
gym_rating_summary	Public-safe aggregates	Public gym profile + app directory

5.4 Members directory

- Members directory is Connected Gym scoped.
- Canonical read: get_tenant_member_directory(tenant_id, limit, cursor) RPC.
- RPC returns can_message flag (computed by can_dm_user + privacy settings) so UI does not guess.
- Follow/unfollow is the social graph; gym membership is separate.

5.5 Events MVP

- events table is tenant-scoped. Staff/admin can create and publish.
- event_rsvps table stores member RSVP state; members can only see their own RSVP rows.
- Event detail counts and visibility are served via RPCs (get_event_detail, get_tenant_events).
- Event share template + calendar export (.ics) are first-class outcomes.

5.6 Inbox announcements (gym-scoped)

- tenant_announcements table stores pinned and archived posts from staff/admin.
- Inbox surfaces announcements in /app/inbox/ (tab).
- Events may auto-create an announcement on publish (optional).
- Management gate: can_manage_announcements(tenant_id) must be staff/admin only.

6. Competitive Layer: Seasons, Challenges, Streaks, Shields, Trophies

6.1 Seasons & Challenges (tenant-scoped, staff-managed)

Challenges are time-boxed seasons that belong to a gym. They are competitive without becoming toxic.

- Challenge lifecycle: Draft → Active → Ended.
- Staff/admin can create seasons (create_challenge_season) or seed a default 30-day season (create_default_30d_challenge).
- Members join (join_challenge) and log daily activity (log_challenge_activity) with strict caps.
- Leaderboards are served by RPC (get_challenge_leaderboard) and must not leak non-member data.

Challenge schema contract

Table	Purpose
challenges	Season record: dates, status, title, rules, tenant_id
challenge_tasks	Task definitions (Train/Recover/Support etc)
challenge_participants	Participant state (points, streaks, shields, joined_at)
challenge_logs	Immutable activity logs (day + task + points)
challenge_streak_shields	Shielded dates (per participant)

6.2 Streaks + Streak Shield

Streaks are motivating, but NDYRA gives users a small escape hatch without making it soft.

- Streak is computed server-side in log_challenge_activity.
- Shield allowance is stored on challenge_participants (streak_shields_total, streak_shields_used).
- use_streak_shield(challenge_id, day) can protect yesterday (UTC) only; must prevent retro-abuse.
- Timezone-friendly streaks: user_local_date() and safe_timezone() handle user tz reliably.

6.3 Badges + Trophy cabinet

- badges defines badge metadata (key, title, description, rarity, icon paths).
- user_badges records awards with source_type/source_id/meta and awarded_at.
- Trophy cabinet visibility: privacy_settings.trophies_visibility + per-trophy override user_badges.visibility.
- Canonical read: get_trophy_cabinet(target_user_id) which respects blocks + follow relationship.

6.4 Recap + share loop

- Challenge recap card includes both Hardcore + Zen metrics: rank, points, best streak, days active, shields left, next badge hint.
- Share templates produce IG story-ready exports; never expose private membership info.

7. Messaging Layer: Inbox, Announcements, DMs, Privacy, Safety

7.1 Inbox as the unified communication surface

- Inbox is the member's 'communications center' for their Connected Gym and their social graph.
- It must provide clear separation: Announcements (gym) vs Direct (DM).
- All unread counts are server-derived; no client guessing.

7.2 Gym Announcements

- tenant_announcements are created by staff/admin (create_announcement RPC).
- Announcements can be pinned/unpinned and archived; members never see archived by default.
- Announcements respect tenant visibility rules (members/staff/public when appropriate).

7.3 Direct Messages (1:1)

DMs are real. They must be safe and spam-resistant by design.

- Threads are 1:1 (dm_threads). Messages live in dm_messages; reads in dm_thread_reads.
- Participant-only RLS: only thread participants can read/write.
- Unread counts must be computed server-side (get_my_dm_threads).
- New messages emit notifications (notification_type='message').

DM request model (anti-spam)

- Trust gates: mutual follow OR shared gym membership OR staff→member OR platform admin.
- If not trusted but recipient allows broader DMs, create requested threads (status='requested') instead of active.
- Recipient can accept_dm_request / decline_dm_request; declines write dm_denials cooldown (14 days).
- Sender can withdraw_dm_request.

Attachments

- DM attachments are images only for now (dm_messages.media_type/media_path/width/height).
- Attachments should store paths in Supabase Storage; do not embed base64 blobs in DB.
- Client should compress images before upload; enforce size limits at upload.

7.4 Privacy controls

- privacy_settings.dm_allow: off | mutual | gym | mutual_or_gym | anyone.
- can_dm_user(target_user) must respect recipient policy; blocks override everything.
- privacy_settings.show_online_status controls Active Now visibility.
- privacy_settings.trophies_visibility controls trophy cabinet visibility.

7.5 Safety actions

- Block user: insert into blocks. This must hide both users across NDYRA (posts, DMs, directories).
- Report conversation: write to reports with proper reason; do not expose reporter identity to other user.
- Staff tooling should later include moderation queues (not in scope of current build wiring).

8. Ops Layer: Waivers, Tokens, Booking, Check-in, Migration

8.1 Waivers (system of record)

- waiver_templates: tenant-scoped waiver text + versioning.
- waiver_signatures: user signatures tied to a waiver template version.
- Gate: has_signed_current_waiver(tenant_id) must be true for check-in / booking (when enabled).
- Signing: sign_current_waiver(tenant_id) RPC writes immutable signature rows.

8.2 Tokens / credits (gym-scoped)

- token_wallets: user balance per tenant.
- token_transactions: immutable ledger for credits/debits.
- credit_tokens(tenant_id, user_id, amount, reason) is staff/admin only.
- spend_tokens(tenant_id, amount, reason) is member-side but validated (no negative balances).

8.3 Scheduling + booking

- class_types define the class products.
- class_sessions define scheduled occurrences.
- class_bookings store member bookings with status enum class_booking_status.
- Booking fork exists: book_class_with_membership vs book_class_with_tokens.

8.4 Check-in

- Check-in must enforce: membership status OR tokens + waiver + any local rules.
- Overrides exist for staff (checkin_overrides) and must be audit logged.
- Biz check-in surface is the operational cockpit; member check-in is light.

8.5 Migration tooling

- migration_batches store import jobs and outcomes.
- tenant-migration-import Netlify function handles CSV ingestion safely (service role key required).
- waiver-template-update Netlify function supports template lifecycle updates.
- Migration is a controlled process: import → verify → commit → cutover. Never silent overwrite.

9. Data Model + Security: Tables, Enums, RPCs, RLS Contracts

9.1 Security model (non-negotiable)

- Row Level Security (RLS) is enabled on all public tables.
- Clients (anon/authenticated) get minimal direct privileges. Writes that require logic go through SECURITY DEFINER RPCs.
- Visibility rules are centralized: can_view_post(post_id) is the gate for posts and their related tables.
- Blocks are global: if is_blocked_between(a,b) is true, hide across feeds, directories, and DMs.
- Never add permissive 'select true' policies to 'make the UI work.' Fix the contract instead.

9.2 Canonical enums

Enum	Values (current)
post_visibility	public, followers, members, staff, private
post_kind	post, signal, aftermath
reaction_type	fire, muscle, bolt, clap, brain (extensible)
notification_type	system, social, message (extensible)
membership_status	active, paused, past_due, comp, cancelled (tenant-defined)
challenge_status	draft, active, ended
class_booking_status	booked, cancelled, waitlist (extensible)
class_visibility	public, members, hidden
report_reason	spam, harassment, hate, sexual, violence, self_harm, other (policy-controlled)

9.3 Table catalogue (build to date)

This catalogue is informational. The canonical implementation is the SQL migrations.

Domain	Tables
Social Core	posts, post_media, post_reactions, post_comments, post_stats, follows_users, follows_tenants, privacy_settings, blocks, reports, notifications
Signals/Aftermath	(stored on posts via kind + expires_at + pinned_to_profile + signal_* columns)
Gyms/Ops	gym_memberships, waiver_templates,

	waiver_signatures, token_wallets, token_transactions, class_types, class_sessions, class_bookings, checkin_overrides, migration_batches
Constellation	gym_ratings, gym_rating_history, gym_rating_summary
Challenges	challenges, challenge_tasks, challenge_participants, challenge_logs, challenge_streak_shields
Badges/Trophies	badges, user_badges
Events	events, event_rsvps
Inbox	tenant_announcements
Direct Messages	dm_threads, dm_messages, dm_thread_reads, dm_denials
Audit	audit_log

9.4 RPC catalogue (high-level)

RPCs are how NDYRA enforces invariants. If an action has rules, it belongs in an RPC.

Domain	RPCs
Social	get_following_feed, can_view_post, can_comment_now, apply_reaction_delta, apply_comment_delta
Signals	get_signal_strip, enforce_signal_limits (trigger), set_connected_tenant (context)
Constellation	can_rate_tenant, get_gym_rating_feedback, get_gym_rating_trend, refresh_gym_rating_summary
Challenges	get_active_challenges, create_default_30d_challenge, create_challenge_season, join_challenge, get_challenge_tasks, log_challenge_activity, get_challenge_leaderboard, get_my_challenge_rank, end_challenge_season, use_streak_shield

Trophies	award_badge, get_trophy_cabinet
Events	get_tenant_events, get_event_detail, create_event, rsvp_event
Inbox	create_announcement, can_manage_announcements
DMs	can_dm_user, start_dm_thread, get_my_dm_threads, send_dm_message, mark_dm_thread_read, accept_dm_request, decline_dm_request, withdraw_dm_request, hide_dm_thread, clear_dm_thread
Ops	has_signed_current_waiver, sign_current_waiver, credit_tokens, spend_tokens, book_class_with_membership, book_class_with_tokens
Admin	is_platform_admin, lookup_user_id_by_email (limited)

9.5 Storage buckets (recommended)

- avatars: profile images
- posts: post media (images now, video later)
- signals: signal videos/images (can be shared bucket with posts if desired)
- badges: badge SVGs (currently shipped as static assets; later could move to storage)
- dm: DM attachment images

10. Infra Wiring: Git, Supabase, Netlify, Stripe (step-by-step)

10.1 Wiring order (do in this order)

16. Freeze the repo at a checkpoint tag (e.g., cp75) and protect main.
17. Create/configure Supabase project, apply migrations, verify RLS.
18. Create/configure Netlify site, connect repo, set env vars, validate rewrites.
19. Create/configure Stripe products + portal + webhooks, connect secrets into Netlify.
20. Run end-to-end smoke: auth → feeds → gym join → challenges → events → inbox → DM → checkout.

10.2 Git setup (authoritative source control)

- Repo lives as a single canonical remote (GitHub or equivalent). No “side repos” for functions or schema.
- Branching: main (protected) + dev (optional) + short-lived feature branches.
- Require PR checks: qa:smoke + qa:super + qa:brand + qa:ip (minimum).
- Tag releases: CPxx tags or release branches; artifacts must map 1:1 to tags.

Recommended minimal GitHub settings:

- Branch protection on main: require status checks, require linear history, block force-push.
- CODEOWNERS for /supabase and /netlify/functions to prevent accidental drift.
- Secrets: never commit .env; store secrets in Netlify + GitHub secrets only.

10.3 Supabase wiring (database + auth + storage)

Supabase is the law layer: it owns identity, permissions, and all privacy enforcement.

Create project

21. Create a new Supabase project (production) in your desired region.
22. Record: Project URL, anon key, service role key.
23. Enable required extensions (uuid-ossp/pgcrypto) if not already enabled.
24. Turn on Realtime for required tables (presence + notifications, if used).

Apply schema

Use one of these two methods (choose one and stay consistent):

- Supabase CLI (preferred): supabase link → supabase db push (applies migrations in order).
- SQL Editor: paste migrations sequentially (risky; only do if CLI is blocked).

After applying schema, run the anti-drift audits in /supabase/gates to confirm the expected invariants are present.

Auth settings

- Email auth enabled (magic link or password, per your policy).
- Redirect URLs include Netlify preview + production domains and localhost (for QA).
- Set secure email templates and do not leak environment names.

Storage buckets

- Create buckets: avatars, posts, dm (minimum).

- Bucket policies: only authenticated users can write; reads depend on post visibility logic (usually mediated via signed URLs or gated storage rules).
- Keep maximum object size constraints (e.g., 10–25MB) until video strategy is finalized.

10.4 Netlify wiring (hosting + functions)

Create Netlify site and connect repo

25. Create a new Netlify site from Git.
26. Build command: npm run build (safe no-op).
27. Publish directory: site
28. Functions directory: netlify/functions
29. Node version: 18+ (20 recommended).

Environment variables (production context)

Set these in Netlify site settings (never commit them):

- VITE_SUPABASE_URL (or SUPABASE_URL)
- VITE_SUPABASE_PUBLISHABLE_KEY (or SUPABASE_ANON_KEY)
- SUPABASE_SERVICE_ROLE_KEY (server-only; required for migration utilities + Stripe webhooks)
- STRIPE_SECRET_KEY
- STRIPE_WEBHOOK_SIGNING_SECRET
- STRIPE_PORTAL_CONFIGURATION_ID (optional but recommended)
- PRICE_ID_* variables (member + biz plan price IDs) OR provide stripe_public_test.json for local-only tests

Redirects + headers

- /site/_redirects must be deployed (Netlify reads it automatically).
- /api/* rewrite must point to /.netlify/functions/:splat.
- Dynamic route rewrites must exist for /app/post/*, /app/profile/*, /app/u/*, /app/signals/*, /gym/*, /workouts/*, /biz/*, /admin/*.
- Headers must set correct caching for build.json and SW files.

Function routing standardization (critical)

CP75 includes functions whose filenames do not match the frontend's expected /api/stripe/* paths. Before wiring Stripe, make routes deterministic.

- Preferred approach: create nested function folders so endpoints match URLs, e.g. netlify/functions/stripe/create-checkout-session.js → /api/stripe/create-checkout-session.
- Alternative approach: add explicit _redirects rules mapping /api/stripe/create-checkout-session → /.netlify/functions/stripe_create_checkout_session, etc.
- Do the same for telemetry endpoint (frontend expects /.netlify/functions/telemetry but function is telemetry_ingest).

10.5 Stripe wiring (payments + billing portal)

Create products + prices

- Create products for Member plans and Gym/Biz plans.
- Create prices (monthly + annual). Keep IDs stable; store them in Netlify env vars.
- Use metadata on products/prices to tag: ndyra_plan_type, ndyra_tier, environment.

Customer portal

- Enable Stripe Customer Portal in Stripe settings.
- Create a portal configuration and store its ID in STRIPE_PORTAL_CONFIGURATION_ID.
- Ensure portal allows: update payment method, cancel subscription, view invoices.

Webhooks

30. Create a Stripe webhook endpoint pointing to your Netlify function route (recommended: /api/stripe/webhook).
31. Subscribe to events: checkout.session.completed, customer.subscription.created/updated/deleted, invoice.paid, invoice.payment_failed.
32. Copy signing secret and set STRIPE_WEBHOOK_SIGNING_SECRET in Netlify.
33. Verify webhook handler writes subscription state to Supabase (service role key required).

End-to-end validation

- Trigger test checkout from /pricing and confirm redirect to Stripe.
- Complete checkout and confirm webhook writes expected rows/flags in Supabase.
- Open billing portal from /app/account or /biz/account and confirm session opens.
- Cancel subscription in portal and confirm webhook updates state.

11. QA Gates + Release Discipline

11.1 Local QA run (from Downloads folder, Windows-friendly)

The fastest correct way to QA locally is to use the repo's QA server (it handles redirects correctly).

34. Unzip the Full Repo Replace zip into a normal folder (no spaces if you can help it).
35. Open a terminal in that folder.
36. Run: npm install
37. Run: npm run start:qa
38. Open: http://localhost:4173
39. Use /auth/qa.html for local QA shortcuts (demo mode, quick nav, cache reset).

If icons/colors look missing: hard refresh (Ctrl+F5) and clear site data (service worker + cache). The QA page includes a one-click cache reset helper.

11.2 Mandatory gates before moving to the next checkpoint

- qa:smoke — confirms build stamp, core files, route integrity.
- qa:super — deeper integrity checks.
- qa:brand — enforces locked NDYRA red (#E10600) and brand tokens.
- qa:ip — checks for competitor asset drift and license hygiene patterns.
- RLS audit SQL — run /supabase/gates/* audits when schema changes.
- E2E tests — Playwright suite passes (desktop + mobile).

11.3 Release packaging discipline

Every checkpoint produces artifacts that map 1:1 to the repo state.

- Full Repo Replace zip (always).
- QA Preview site zip (site folder only, for fast preview).
- Buildbook Kit zip (scripts + runbooks + memory pack).
- Master Deliverables is split:
- • Part 1: Repo + all Buildbook artifacts (kept together)
- • Part 2: Everything else (QA reports, preview, manifests, etc.)
- No patch zips going forward — incorporate changes into the repo so the build is clean.

11.4 'Thorough QA before continue' rule

Before any new feature build begins, we run the full QA on the previous checkpoint and fix anything that fails. This prevents compounding drift and saves time downstream.

Appendix A: Environment Variables + Secrets Map

Variable	Used by	Where set	Sensitivity
VITE_SUPABASE_URL	public_config function → frontend	Netlify env (public)	Public
VITE_SUPABASE_PUBLISHABLE_KEY	public_config function → frontend	Netlify env (public)	Public
SUPABASE_SERVICE_ROLE_KEY	Stripe webhooks, migration utilities	Netlify env	Secret
SUPABASE_URL	Server + public fallback	Netlify env	Public (URL)
SUPABASE_ANON_KEY	Public fallback	Netlify env	Public
STRIPE_SECRET_KEY	Stripe functions	Netlify env	Secret
STRIPE_WEBHOOK_SIGNING_SECRET	stripe_webhook verification	Netlify env	Secret
STRIPE_PORTAL_CONFIGURATION_ID	stripe_create_portal_session	Netlify env	Not secret
PRICE_ID_MEMBER_MONTHLY / PRICE_ID_MEMBER_ANNUAL / PRICE_ID_BIZ_*	stripe_create_checkout_session	Netlify env	Not secret
TELEMETRY_WEBHOOK_URL	telemetry_ingest passthrough (optional)	Netlify env	Secret/Optional
URL / DEPLOY_PRIME_URL / CONTEXT	Netlify runtime	Auto	N/A

Rule: only Supabase URL + anon key are allowed to reach the browser. Service role keys must never be returned by public_config or embedded in site JS.

Appendix B: Netlify redirects/headers requirements

The static app relies on rewrites so that deep links work. These must exist in production and in local QA server behavior.

Minimum required patterns (illustrative — keep in sync with /site/_redirects):

```
# API → Functions
/api/*  /.netlify/functions/:splat  200

# App deep links
/app/*            /app/index.html            200
/app/post/*       /app/post/index.html       200
/app/profile/*    /app/profile/index.html    200
/app/u/*          /app/u/index.html          200
/app/signals/*    /app/signals/index.html    200
/app/book/class/* /app/book/class/index.html 200

# Gym deep links
/gym/*            /gym/profile/index.html    200
/gym/*/join       /gym/join/index.html       200

# Biz/Admin deep links
/biz/*            /biz/index.html            200
/admin/*          /admin/index.html          200
```

Headers must also ensure the service worker and build.json do not get cached incorrectly during QA. Keep cache-control conservative for build stamps and SW.

Appendix C: Known drifts found in CP75 + required fixes

These are concrete mismatches observed in the repo state (CP75) that will block real infra wiring if not corrected.

- Stripe endpoint mismatch: frontend calls /api/stripe/create-checkout-session but the function filename is stripe_create_checkout_session (so Netlify expects /.netlify/functions/stripe_create_checkout_session). Fix via nested function folders or explicit redirects.
- Stripe portal + webhook paths should be standardized the same way (/api/stripe/create-portal-session and /api/stripe/webhook).
- Telemetry endpoint mismatch: frontend expects /.netlify/functions/telemetry but function is telemetry_ingest. Decide one canonical name and align both.
- Telemetry function still contains legacy naming strings (HIIT56). Rename to NDYRA for cleanliness.
- If local QA shows missing icons/colors: ensure service worker cache reset and hard refresh are part of QA checklist (they are now).

Once these are fixed and deployed, the Agent wiring plan in Section 10 can be executed without surprises.